

Análisis comparativo entre microservicios y monolito modular aplicado al sistema TiendaTech

TA-IND-02 — Informe académico individual — Aplicaciones Distribuidas

Andy Paul Sánchez Pilalao · Quevedo, 13/06/2026

Resumen

La selección de un estilo arquitectónico condiciona el rendimiento, la mantenibilidad, la escalabilidad y la complejidad operativa de un sistema distribuido. Este informe presenta un análisis comparativo entre la arquitectura de microservicios y el monolito modular, aplicado al sistema TiendaTech, una plataforma de comercio electrónico orientada a la venta de componentes tecnológicos y al ensamblaje asistido de computadoras. La investigación se desarrolló mediante una revisión documental estructurada de nueve publicaciones académicas difundidas entre 2021 y 2026, considerando estudios experimentales, revisiones sistemáticas y reportes de experiencia industrial. La comparación se organizó según los criterios de rendimiento, escalabilidad, mantenibilidad, despliegue, gestión de datos, resiliencia y adecuación al tamaño del equipo.

La evidencia revisada muestra que los microservicios favorecen el despliegue independiente, el escalamiento selectivo y la autonomía de los componentes, pero introducen costos asociados con la comunicación remota, la observabilidad, la consistencia distribuida y la automatización de pruebas. Por otro lado, el monolito modular conserva límites internos entre dominios mientras reduce la infraestructura y la coordinación operativa necesarias. Para la etapa inicial de TiendaTech, el monolito modular representa una alternativa técnicamente proporcionada; sin embargo, una extracción gradual de módulos hacia microservicios puede justificarse cuando existan necesidades verificables de escalamiento, aislamiento de fallos o despliegue independiente.

Palabras clave: arquitectura de software, microservicios, monolito modular, sistemas distribuidos, escalabilidad, mantenibilidad, TiendaTech.

Abstract

The selection of an architectural style determines the performance, maintainability, scalability, and operational complexity of a distributed system. This report presents a comparative analysis between microservices architecture and the modular monolith,

applied to TiendaTech, an e-commerce platform focused on technology components and assisted computer assembly. The study was conducted through a structured documentary review of nine academic publications released between 2021 and 2026, including experimental studies, systematic reviews, and industrial experience reports.

The reviewed evidence indicates that microservices support independent deployment, selective scaling, and component autonomy, but introduce additional costs related to remote communication, observability, distributed consistency, and test automation. In contrast, a modular monolith preserves clear internal domain boundaries while reducing the required infrastructure and operational coordination. For the initial stage of TiendaTech, the modular monolith represents a technically proportionate alternative. Nevertheless, a gradual extraction of modules into microservices may be justified when measurable requirements for scaling, fault isolation, or independent deployment emerge. **Keywords:** software architecture, microservices, modular monolith, distributed systems, scalability, maintainability, TiendaTech.

1. Introducción

Las aplicaciones modernas deben responder a necesidades de crecimiento, disponibilidad, evolución continua y mantenimiento. Como consecuencia, la selección del estilo arquitectónico constituye una de las decisiones de mayor impacto en el ciclo de vida de un sistema de software. Una arquitectura inadecuada puede producir problemas de acoplamiento, sobrecostos de infraestructura, baja capacidad de evolución o una complejidad operativa desproporcionada respecto al problema que se intenta resolver.

Durante los últimos años, la arquitectura de microservicios ha adquirido una amplia aceptación debido a su capacidad para dividir una aplicación en servicios autónomos, desplegables y escalables de manera independiente. Sin embargo, su adopción también introduce dificultades relacionadas con la comunicación entre servicios, la consistencia de los datos, la observabilidad, la automatización de pruebas y la administración de múltiples componentes distribuidos [1], [2], [3].

Frente a esta complejidad, el monolito modular ha surgido como una alternativa que mantiene un único proceso de despliegue, pero organiza la aplicación mediante módulos con responsabilidades y límites explícitos. Su y Li señalan que este estilo intenta combinar la simplicidad operativa del monolito con algunas de las ventajas de modularidad asociadas con los microservicios, e incluso puede constituir una etapa previa a una futura migración [4].

El presente análisis se aplica a TiendaTech, un sistema de comercio electrónico orientado a la venta de hardware y al ensamblaje asistido de computadoras. Su diseño actual contempla servicios de autenticación, catálogo, inventario, carrito, pedidos, pagos y notificaciones. Aunque esta separación permite representar con claridad diferentes capacidades del negocio, también plantea la pregunta de si el alcance inicial, el volumen esperado de usuarios y el tamaño del equipo justifican toda la complejidad de una arquitectura distribuida.

En consecuencia, se plantea la siguiente pregunta de análisis:

¿Cuál de los dos estilos arquitectónicos ofrece el equilibrio más adecuado entre escalabilidad, mantenibilidad, autonomía de despliegue y complejidad operativa para la etapa inicial del sistema TiendaTech?

El objetivo general del informe es comparar críticamente la arquitectura de microservicios y el monolito modular, identificando sus ventajas, limitaciones y condiciones de aplicación, con el propósito de fundamentar una decisión arquitectónica para TiendaTech.

Para alcanzar este objetivo se establecieron los siguientes objetivos específicos:

1. Describir las características fundamentales de los microservicios y del monolito modular.

2. Comparar ambos estilos según rendimiento, escalabilidad, mantenibilidad, despliegue, resiliencia y gestión de datos.
3. Analizar las implicaciones de cada alternativa para un equipo de desarrollo pequeño.
4. Proponer una estrategia arquitectónica técnicamente justificada para la evolución de TiendaTech.

El resto del documento se estructura de la siguiente manera. La Sección 2 describe el método utilizado para seleccionar y analizar la literatura. La Sección 3 desarrolla los fundamentos teóricos de ambos estilos. La Sección 4 presenta la comparación técnica. La Sección 5 aplica los resultados a TiendaTech. La Sección 6 discute las compensaciones y limitaciones encontradas, y finalmente la Sección 7 presenta las conclusiones técnicas.

2. Metodología

El trabajo siguió un enfoque de revisión narrativa estructurada. Se recopilaron inicialmente once publicaciones académicas relacionadas con microservicios, monolitos, monolitos modulares, migración arquitectónica, persistencia distribuida, mantenibilidad, desempeño y despliegue. Después de evaluar su pertinencia, actualidad y disponibilidad de un DOI verificable, se seleccionaron nueve publicaciones para sustentar el análisis final. Las publicaciones seleccionadas fueron difundidas entre 2021 y 2026 en revistas o conferencias de IEEE y ACM, y en todos los casos cuentan con un identificador DOI verificable.

Debido a las restricciones de acceso a determinados textos completos, el análisis se realizó a partir de los metadatos bibliográficos, los resúmenes disponibles y los resultados explícitamente reportados por los autores. No se atribuyeron a los estudios procedimientos, resultados o limitaciones que no estuvieran disponibles en la información consultada.

La selección se realizó en tres etapas. En primer lugar, se buscaron estudios que compararan directamente arquitecturas monolíticas y de microservicios. En segundo lugar, se incorporaron investigaciones sobre problemas específicos de los sistemas distribuidos, como descomposición de aplicaciones, persistencia polígota, calidad arquitectónica y automatización de despliegues. Finalmente, se incluyeron estudios relacionados con equipos pequeños y proyectos colaborativos para conectar la evidencia con el contexto académico de TiendaTech.

Los criterios utilizados para la comparación fueron:

- rendimiento y consumo de recursos;

- escalabilidad vertical y horizontal;
- modularidad y mantenibilidad;
- autonomía y frecuencia de despliegue;
- complejidad operacional;
- consistencia y propiedad de los datos;
- tolerancia a fallos y resiliencia;
- coordinación del equipo de desarrollo;
- facilidad de evolución y migración.

No se realizó una experimentación propia de rendimiento. Por lo tanto, los resultados cuantitativos utilizados en el documento corresponden únicamente a los estudios revisados. Esta delimitación evita generalizar resultados más allá del contexto en el que fueron obtenidos.

3. Marco teórico

3.1. Arquitecturas distribuidas

Una arquitectura distribuida organiza los componentes de una aplicación en procesos o nodos que intercambian información mediante una red. Esta distribución permite separar responsabilidades, escalar componentes y aislar determinados fallos, pero también incorpora incertidumbre en la comunicación, latencia, posibles pérdidas de mensajes y problemas de consistencia.

La distribución no debe considerarse un objetivo por sí misma. Su conveniencia depende de que los beneficios obtenidos compensen los costos de comunicación, despliegue, seguridad, monitoreo y coordinación. En sistemas pequeños o con carga limitada, estos costos pueden superar las ventajas previstas [1], [4].

3.2. Arquitectura de microservicios

La arquitectura de microservicios divide una aplicación en servicios pequeños y autónomos, alineados con capacidades específicas del negocio. Cada servicio puede desarrollar su propia lógica, exponer interfaces bien definidas y desplegarse sin necesidad de publicar nuevamente toda la aplicación [1], [2].

Entre sus principales ventajas se encuentran el escalamiento selectivo, la autonomía tecnológica, el aislamiento parcial de fallos y la posibilidad de que diferentes equipos

desarrollen componentes en paralelo. En proyectos colaborativos, esta separación también puede facilitar la distribución de tareas y responsabilidades [5].

No obstante, la autonomía de los servicios desplaza parte de la complejidad desde el código interno hacia la infraestructura. Los equipos deben administrar contratos de comunicación, versionado de interfaces, descubrimiento de servicios, trazabilidad distribuida, pruebas de integración, automatización de despliegues y mecanismos de recuperación ante fallos.

Pianini y Neri advierten que una modificación local aparentemente correcta puede producir efectos en cascada sobre otros servicios, incluso cuando cada componente supera sus pruebas individuales. Por esta razón, una arquitectura de microservicios requiere integración continua, pruebas globales y prácticas DevOps suficientemente maduras [2].

3.3. Monolito modular

El monolito modular conserva una única unidad de ejecución y despliegue, pero divide internamente la aplicación en módulos con responsabilidades, dependencias y contratos explícitos. A diferencia de un monolito tradicional desorganizado, la comunicación entre módulos debe respetar límites arquitectónicos y evitar el acceso indiscriminado a la implementación interna de otras áreas.

Su y Li identifican al monolito modular como una alternativa a los microservicios que intenta conservar ventajas de ambos estilos. El sistema mantiene una operación y despliegue relativamente simples, pero utiliza límites modulares que pueden facilitar una futura extracción de servicios [4].

El monolito modular no elimina la necesidad de diseño. Una implementación que solo agrupa clases en carpetas, pero permite dependencias circulares y acceso directo a todas las tablas, continúa siendo un monolito fuertemente acoplado. Por lo tanto, se requieren límites de dominio, interfaces internas, reglas de dependencia y pruebas que impidan la degradación de la modularidad.

3.4. Evolución desde un monolito hacia microservicios

La migración hacia microservicios implica identificar capacidades de negocio, analizar dependencias y determinar qué partes pueden evolucionar de forma independiente. Abgaz et al. encontraron que la descomposición de aplicaciones monolíticas todavía carece de herramientas, métricas, conjuntos de datos y líneas base suficientemente estandarizadas [6].

De forma complementaria, Oumoussa y Saidi destacan que la identificación correcta de microservicios sigue siendo un desafío en evolución. Entre las necesidades pendientes

se encuentran herramientas más precisas, métodos de evaluación comunes y una mejor comprensión de los factores humanos que afectan la transición [7].

Estos resultados respaldan una estrategia gradual: primero establecer módulos cohesivos y límites de dominio, y posteriormente extraer únicamente aquellos componentes cuya independencia aporte beneficios verificables.

4. Análisis comparativo

4.1. Rendimiento

En un monolito modular, la comunicación entre módulos ocurre generalmente dentro del mismo proceso. Esto evita serialización, llamadas remotas y tránsito por la red. Como resultado, las operaciones que involucran varios módulos pueden ejecutarse con menor latencia y con menos infraestructura.

Blinowski et al. compararon una aplicación monolítica y una aplicación de microservicios implementadas en Java y .NET. En los experimentos ejecutados sobre una sola máquina, la versión monolítica obtuvo un mejor rendimiento que su equivalente distribuido. El estudio también observó que el escalamiento vertical resultó más rentable que el horizontal en los entornos de Azure analizados, y que aumentar el número de instancias más allá de cierto punto podía degradar el desempeño [1].

Estos resultados no implican que los microservicios sean inherentemente ineficientes. Su principal beneficio consiste en permitir que componentes específicos escalen de forma independiente. Sin embargo, esa capacidad solo es ventajosa cuando existen diferencias reales de carga entre los servicios.

4.2. Escalabilidad

Los microservicios permiten asignar recursos adicionales únicamente a los componentes con mayor demanda. Por ejemplo, un servicio de catálogo puede recibir más tráfico que un servicio de notificaciones y escalarse sin replicar toda la aplicación.

En un monolito modular, el escalamiento replica normalmente la aplicación completa. Esta limitación puede producir un uso menos eficiente de recursos en sistemas de gran tamaño. No obstante, para aplicaciones con una carga inicial moderada, el escalamiento vertical o la replicación del monolito completo pueden ser suficientes y más sencillos de administrar.

Por tanto, la escalabilidad debe analizarse según la carga esperada y no como una característica abstracta. Adoptar microservicios para resolver un problema de escala que todavía no existe puede aumentar la complejidad sin generar un beneficio inmediato [1], [4].

4.3. Mantenibilidad y calidad arquitectónica

Los dos estilos pueden ser mantenibles si conservan límites claros y responsabilidades cohesionadas. La diferencia principal está en el mecanismo utilizado para hacer cumplir esos límites.

En los microservicios, la separación mediante procesos y contratos de red impide cierto tipo de acceso directo entre componentes. Sin embargo, pueden aparecer dependencias distribuidas, cadenas extensas de llamadas y acoplamiento a nivel de mensajes o interfaces.

Makieiev y Kravets proponen evaluar continuamente atributos como rendimiento, escalabilidad, confiabilidad, mantenibilidad y modularidad mediante funciones de aptitud arquitectónica. Su estudio demuestra que una arquitectura de microservicios necesita mecanismos explícitos para detectar cambios y degradaciones de calidad [8].

En el monolito modular, los límites dependen en mayor medida de reglas de diseño, revisiones de código y pruebas arquitectónicas. Su ventaja es que la refactorización y la depuración pueden realizarse dentro de un entorno único, sin necesidad de rastrear solicitudes entre múltiples procesos.

4.4. Gestión y consistencia de datos

Un monolito modular puede utilizar una única base de datos y ejecutar transacciones locales entre módulos. Esta característica simplifica operaciones que deben modificar varias entidades de forma atómica.

En microservicios se recomienda que cada servicio sea propietario de sus datos. Esta autonomía evita que otros servicios dependan directamente de sus tablas, pero convierte algunas operaciones en transacciones distribuidas. En tales casos pueden ser necesarios patrones como Saga, transactional outbox o event sourcing.

Halili et al. concluyen que la persistencia políglota puede mejorar la adaptabilidad, el rendimiento y la alineación con el dominio, pero aumenta la complejidad operativa y de gobierno. La selección de diferentes tecnologías de datos debe justificarse por las necesidades concretas de cada servicio y no por la simple posibilidad de utilizar múltiples motores [3].

4.5. Resiliencia y aislamiento de fallos

La separación de componentes en procesos independientes puede limitar el impacto de determinados fallos, debido a que la caída de un servicio no implica necesariamente la interrupción completa del sistema. No obstante, esta ventaja depende de que existan mecanismos explícitos de tolerancia a fallos, recuperación y control de dependencias.

Cuando estos controles no están presentes, una dependencia no disponible puede producir fallos en cascada entre varios servicios [2], [9].

En un monolito modular, los módulos comparten el mismo proceso y, por tanto, un fallo no controlado puede comprometer la aplicación completa. A cambio, la comunicación interna evita problemas asociados con la red y permite manejar determinadas transacciones y excepciones dentro de una misma unidad de ejecución. En consecuencia, los microservicios ofrecen un mayor potencial de aislamiento, pero requieren infraestructura y patrones adicionales para materializarlo.

4.6. Despliegue y operación

La principal ventaja operativa de los microservicios es la capacidad de publicar cambios de forma independiente. No obstante, esta autonomía requiere integración continua, compatibilidad de contratos y mecanismos de observación del sistema completo [2], [9].

Dhawan presenta un caso en el que una arquitectura desacoplada entre gateway y microservicios redujo el tiempo de despliegue desde cuatro semanas hasta menos de diez minutos. La prueba de concepto procesó más de treinta mil solicitudes y mantuvo mecanismos de gobierno centralizados en el gateway [9].

Sin embargo, el propio estudio reconoce que sus resultados corresponden a una prueba de concepto y no pueden generalizarse automáticamente. Además, obtener despliegues independientes exige pipelines, registro de imágenes, configuración externa, observabilidad y estrategias de reversión.

En un monolito modular existe un solo artefacto de despliegue. Esta característica reduce el número de pipelines y entornos, pero cualquier cambio requiere publicar la aplicación completa.

4.7. Comparación general

En síntesis, los microservicios trasladan parte de la complejidad desde la estructura interna del código hacia la coordinación entre procesos, infraestructura y contratos distribuidos. El monolito modular mantiene dicha coordinación dentro de una sola unidad de ejecución, aunque exige disciplina arquitectónica para evitar el acoplamiento entre módulos.

Por lo tanto, la elección no debe basarse únicamente en cuál alternativa ofrece más capacidades, sino en cuáles de esas capacidades son realmente necesarias. Para equipos pequeños, la simplicidad operativa puede tener mayor valor que la autonomía de despliegue; en sistemas con cargas diferenciadas y equipos independientes, el equilibrio puede favorecer a los microservicios. La Tabla 1 resume estas compensaciones.

Cuadro 1: Comparación entre microservicios y monolito modular

Criterio	Microservicios	Monolito modular
Unidad de despliegue	Varios servicios desplegados de forma independiente.	Una sola aplicación desplegable.
Comunicación	Llamadas REST, mensajería o eventos mediante red.	Invocaciones internas entre módulos.
Escalabilidad	Escalamiento selectivo por servicio.	Escalamiento de la aplicación completa.
Rendimiento	Puede incorporar latencia, serialización y saltos de red.	Menor sobrecarga para operaciones internas.
Datos	Propiedad descentralizada y posible persistencia polígloa.	Base compartida o esquemas internos controlados.
Consistencia	Puede requerir sagas, eventos y compensaciones.	Permite transacciones locales más simples.
Tolerancia a fallos	Puede aislar fallos por servicio, aunque existe riesgo de propagación entre dependencias.	Los módulos comparten un proceso, por lo que un fallo grave puede afectar a toda la aplicación.
Despliegue	Mayor autonomía, pero más infraestructura y automatización.	Despliegue simple, aunque afecta a toda la aplicación.
Observabilidad	Requiere trazas, métricas y registros distribuidos.	Monitoreo y depuración centralizados.
Equipo	Favorece autonomía cuando existen equipos independientes.	Adecuado para equipos pequeños con coordinación directa.
Evolución	Permite reemplazar servicios, pero exige compatibilidad de contratos.	Facilita refactorización interna y extracción gradual de módulos.

Fuente: elaboración propia a partir de [1], [2], [3], [4], [8], [9].

5. Aplicación al sistema TiendaTech

TiendaTech contempla capacidades de autenticación, catálogo, compatibilidad de componentes, inventario, carrito, pedidos, pagos y notificaciones. Estas capacidades pueden organizarse tanto como servicios distribuidos como mediante módulos internos claramente delimitados.

En una arquitectura de microservicios, una posible organización sería:

- *Auth Service*: autenticación y autorización;
- *Catalog Service*: productos, categorías y compatibilidad;
- *Inventory Service*: existencias y reservas;

- *Cart Service*: carrito temporal del comprador;
- *Order Service*: generación y seguimiento de pedidos;
- *Payment Service*: integración con proveedores de pago;
- *Notification Service*: correos y avisos del sistema.

Esta separación favorece la claridad de responsabilidades y permite que servicios como Catálogo o Notificaciones evolucionen de manera independiente. Sin embargo, también crea dependencias distribuidas. Por ejemplo, confirmar una compra requiere coordinar pedido, inventario, pago y notificación. Un fallo en cualquiera de estos componentes puede dejar el proceso en un estado intermedio.

En un monolito modular, las mismas capacidades pueden implementarse como módulos dentro de una aplicación Spring Boot. Cada módulo tendría su propia capa de dominio, aplicación, infraestructura e interfaces internas. La base de datos podría mantenerse compartida durante la etapa inicial, restringiendo el acceso de cada módulo a sus propias tablas o repositorios.

Para un equipo universitario pequeño, esta alternativa reduce la cantidad de contenedores, pipelines, configuraciones, puntos de fallo y contratos remotos que deben administrarse. Al mismo tiempo, conserva una estructura que puede permitir una futura extracción de servicios.

No todos los módulos poseen la misma necesidad de independencia. En TiendaTech, los primeros candidatos para una eventual extracción serían:

1. **Notificaciones**, debido a que puede operar de forma asíncrona y tolerar retrasos.
2. **Pagos**, porque integra un proveedor externo y requiere controles de seguridad y disponibilidad específicos.
3. **Catálogo**, si el volumen de consultas llega a superar ampliamente el de las operaciones de escritura.
4. **Recomendación o ensamblaje asistido**, debido a que podría requerir recursos y tecnologías diferentes al resto del sistema.

En cambio, Pedidos e Inventario presentan una relación transaccional estrecha. Separarlos prematuramente obligaría a introducir reservas, eventos, compensaciones y mecanismos de consistencia eventual antes de que exista una necesidad comprobada.

“`latex`

Repositorio del proyecto

El código fuente, la documentación técnica y los recursos asociados al sistema TiendaTech se encuentran disponibles en el siguiente repositorio:

<https://github.com/AndySanchez2004/TA-IND-01>

6. Discusión

La literatura revisada muestra que los microservicios no constituyen una evolución obligatoria de todo sistema monolítico. Su conveniencia depende de factores técnicos y organizacionales como carga, frecuencia de cambios, madurez de DevOps, número de equipos, autonomía requerida y heterogeneidad tecnológica.

Los resultados de Blinowski et al. contradicen la idea de que distribuir una aplicación mejora automáticamente su rendimiento. En condiciones de una sola máquina, el monolito evaluado obtuvo mejores resultados, y el escalamiento horizontal no produjo beneficios ilimitados [1]. Esto es relevante para TiendaTech, porque todavía no existe evidencia de que el sistema requiera escalar diferentes componentes de forma independiente.

Por otra parte, el caso presentado por Dhawan demuestra que la autonomía de despliegue puede generar mejoras importantes cuando existe una infraestructura estable y servicios que evolucionan con alta frecuencia [9]. No obstante, esta ventaja depende de contar con automatización, contratos estables y mecanismos de observabilidad.

Las revisiones de Abgaz et al. y Oumoussa y Saidi también muestran que la descomposición correcta de un monolito continúa siendo un problema abierto [6], [7]. En consecuencia, diseñar primero límites modulares permite observar las dependencias reales antes de convertir cada capacidad en un proceso distribuido.

Desde una perspectiva académica, la arquitectura de microservicios continúa siendo valiosa para TiendaTech porque permite aplicar comunicación distribuida, API REST, mensajería, contenerización y patrones de resiliencia. Sin embargo, desde una perspectiva de producto, el alcance inicial y el tamaño del equipo favorecen una solución más simple.

A partir de la evidencia analizada, la decisión más equilibrada no consiste en rechazar completamente los microservicios, sino en adoptar una evolución progresiva. TiendaTech puede diseñarse con límites de dominio compatibles con microservicios, mientras evita distribuir desde el inicio aquellos módulos que no presentan necesidades independientes de escalamiento o despliegue.

Entre las principales limitaciones de este análisis se encuentran la ausencia de experimentos propios sobre TiendaTech, la utilización de estudios con diferentes tecnologías y cargas, y la limitada cantidad de investigaciones que comparan específicamente microservicios con monolitos modulares. Por lo tanto, la recomendación deberá validarse

posteriormente mediante mediciones de carga, latencia, consumo de recursos y frecuencia de despliegue.

7. Conclusiones técnicas

La comparación permitió determinar que la arquitectura de microservicios y el monolito modular responden a necesidades diferentes, por lo que ninguno puede considerarse superior en todos los escenarios.

La evidencia revisada mostró que los microservicios ofrecen mayor autonomía de despliegue, escalamiento selectivo y aislamiento de responsabilidades. Sin embargo, estas capacidades incorporan costos relacionados con la comunicación remota, la consistencia distribuida, la observabilidad, la automatización y la coordinación operativa.

La literatura analizada también indicó que el monolito modular permite mantener límites claros entre las capacidades del negocio mediante una infraestructura más simple. Esta alternativa resulta especialmente pertinente para sistemas en etapa inicial, con cargas moderadas y equipos pequeños, siempre que los límites entre módulos se controlen mediante interfaces y reglas arquitectónicas explícitas.

Para una primera versión productiva de TiendaTech con alcance limitado, el monolito modular representa la alternativa más proporcionada desde una perspectiva técnica y operativa. No obstante, para el contexto académico del PFC se mantiene la pertinencia de los microservicios, debido a que permiten aplicar comunicación distribuida, contenerización, APIs independientes, mensajería y mecanismos de resiliencia.

La evolución de TiendaTech debería realizarse de forma gradual y sustentarse en evidencia. La saturación independiente de un componente, una frecuencia de despliegue elevada, requisitos particulares de disponibilidad, integraciones externas o la necesidad de tecnologías especializadas constituirían criterios válidos para extraer determinados módulos como microservicios.

En consecuencia, se respondió la pregunta de análisis estableciendo que el monolito modular ofrece inicialmente el mejor equilibrio entre mantenibilidad, rendimiento y complejidad operativa, mientras que los microservicios adquieren mayor conveniencia cuando aparecen necesidades comprobables de escalamiento, aislamiento o autonomía de despliegue.

Declaración de uso de inteligencia artificial

Para la elaboración de este informe se utilizó ChatGPT como herramienta de apoyo para organizar la estructura del documento, revisar la redacción técnica, clasificar las referencias recopiladas y generar una primera versión del código L^AT_EX. La selección

del tema, la búsqueda de fuentes académicas, la revisión de los resúmenes académicos, la aplicación al sistema TiendaTech y las decisiones arquitectónicas fueron revisadas y validadas por el autor. Las afirmaciones principales y las cifras específicas fueron contrastadas con la información disponible en las fuentes consultadas. La herramienta no se utilizó como fuente bibliográfica y las decisiones, interpretaciones y conclusiones fueron revisadas y asumidas por el autor.

Referencias

- [1] G. Blinowski, A. Ojdowska y A. Przybyłek, «Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,» *IEEE Access*, vol. 10, págs. 20357-20374, 2022. DOI: 10.1109/ACCESS.2022.3152803. visitado 13 de jun. de 2026. dirección: <https://doi.org/10.1109/ACCESS.2022.3152803>.
- [2] D. Pianini y A. Neri, «Breaking Down Monoliths with Microservices and DevOps: An Industrial Experience Report,» en *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, IEEE, 2021, págs. 505-514. DOI: 10.1109/ICSME52107.2021.00051. visitado 13 de jun. de 2026. dirección: <https://doi.org/10.1109/ICSME52107.2021.00051>.
- [3] M. K. Halili, F. Halili, D. M. Veliu y A. Nuhiji, «Polyglot Persistence in Microservices: Managing Data Diversity in Distributed Systems,» en *2025 IEEE International Conference on Internet of Things and Intelligence Systems (IoTaIS)*, IEEE, 2025, págs. 247-253. DOI: 10.1109/IoTaIS67227.2025.11282136. visitado 13 de jun. de 2026. dirección: <https://doi.org/10.1109/IoTaIS67227.2025.11282136>.
- [4] R. Su y X. Li, «Modular Monolith: Is This the Trend in Software Architecture?» En *Proceedings of the 1st International Workshop on New Trends in Software Architecture (SATrends '24)*, New York, NY, USA: Association for Computing Machinery, 2024, págs. 10-13. DOI: 10.1145/3643657.3643911. visitado 13 de jun. de 2026. dirección: <https://doi.org/10.1145/3643657.3643911>.
- [5] Y. M. Lau, C. M. Koh y L. Jiang, «Teaching Software Development for Real-World Problems Using a Microservice-Based Collaborative Problem-Solving Approach,» en *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET '24)*, New York, NY, USA: Association for Computing Machinery, 2024, págs. 22-33. DOI: 10.1145/3639474.3640064. visitado 13 de jun. de 2026. dirección: <https://doi.org/10.1145/3639474.3640064>.

- [6] Y. M. Abgaz et al., «Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review,» *IEEE Transactions on Software Engineering*, vol. 49, n.º 8, págs. 4213-4242, 2023. DOI: 10.1109/TSE.2023.3287297. visitado 13 de jun. de 2026. dirección: <https://doi.org/10.1109/TSE.2023.3287297>.
- [7] I. Oumoussa y R. Saidi, «Evolution of Microservices Identification in Monolith Decomposition: A Systematic Review,» *IEEE Access*, vol. 12, págs. 23 389-23 405, 2024. DOI: 10.1109/ACCESS.2024.3365079. visitado 13 de jun. de 2026. dirección: <https://doi.org/10.1109/ACCESS.2024.3365079>.
- [8] O. Makieiev y N. Kravets, «Evaluation of Microservices Architecture Quality Using Fitness Functions,» en *2025 IEEE 13th International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS)*, IEEE, 2025. DOI: 10.1109/IDAACS68557.2025.11322128. visitado 13 de jun. de 2026. dirección: <https://doi.org/10.1109/IDAACS68557.2025.11322128>.
- [9] M. Dhawan, «Decoupling Deployment Velocity From Platform Governance: An Empirical Case Study of WebSocket-Enabled Gateway-Microservice Architecture,» *IEEE Access*, vol. 14, 2026. DOI: 10.1109/ACCESS.2026.3654150. visitado 13 de jun. de 2026. dirección: <https://doi.org/10.1109/ACCESS.2026.3654150>.